



Course Name		
Computer Programming Lab		
Lab Title		
Implementing Tuple, List, and understanding aliasing, cloning		
Name	Registration Number	Lab #
Muhammad Hamzah Iqbal	F24604018	4
Date	Instructor's Name	Instructor's Signature
19/3/2024	LE Ayesha Ashfaque	

Objective:

- To have basic understanding of Tuples and Lists in Python.
- To understand the concept of Aliasing and Cloning in Python.

Theoretical Aspect

- **Lists in Programming:** A list is a collection of items stored in a specific order. Lists are mutable, meaning elements can be changed, added, or removed. Python lists can store different data types, including numbers, strings, and even other lists.
- **Creation & Access:** Lists are defined using [], accessed via indexing (positive and negative).
- **Modification & Operations:** Elements can be added (append(), insert()), removed (remove(), pop()), and modified. Lists support sorting, reversing, and slicing.
- **Looping & Iteration:** Lists can be processed using for and while loops for efficient data handling.

Experimental Procedure

Task 01:

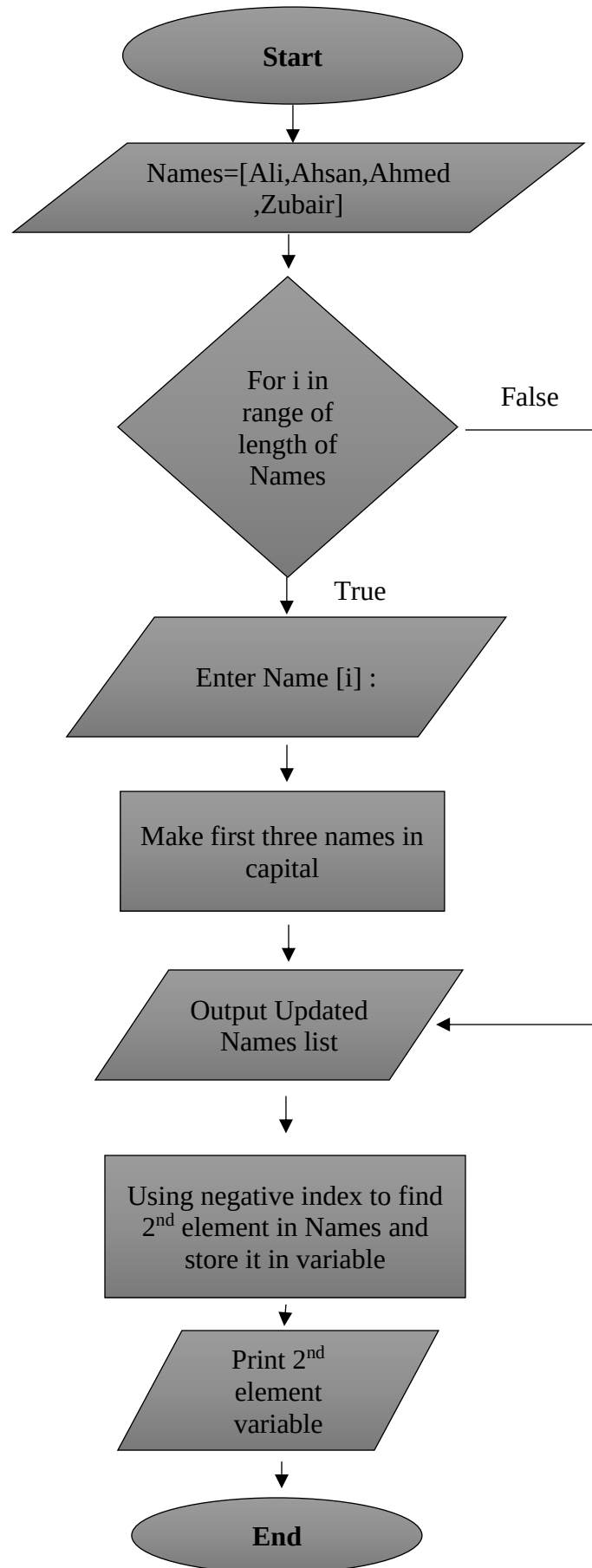
Make this list, Names = ['Ali', 'Ahsan', 'Ahmed', 'Zubair']. Print list and then overwrite each element of this list by asking user for input. Make sure first three members of the list should be uppercased and print again. Find the name at index 2 using negative indexing.

Solution:

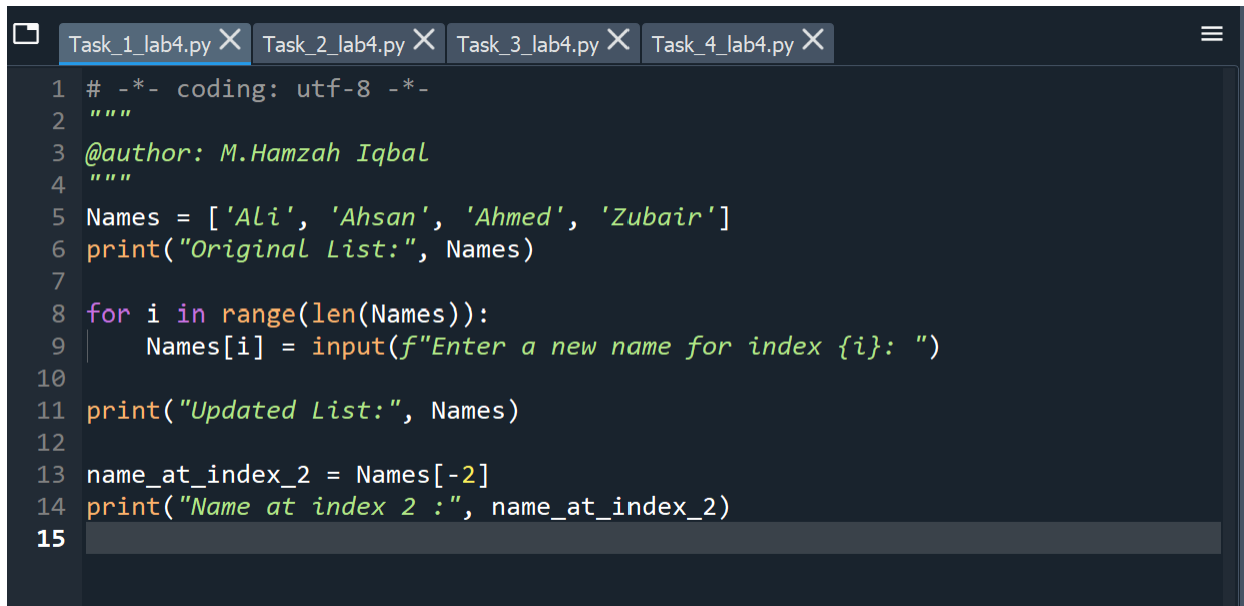
Algorithm:

1. Initialize a list Names with ['Ali', 'Ahsan', 'Ahmed', 'Zubair'].
2. Print the original list.
3. Loop through each index of the list:
 - Input a new name for that index.
4. Print the updated list.
5. Retrieve and print the name at index 2

Flowchart:

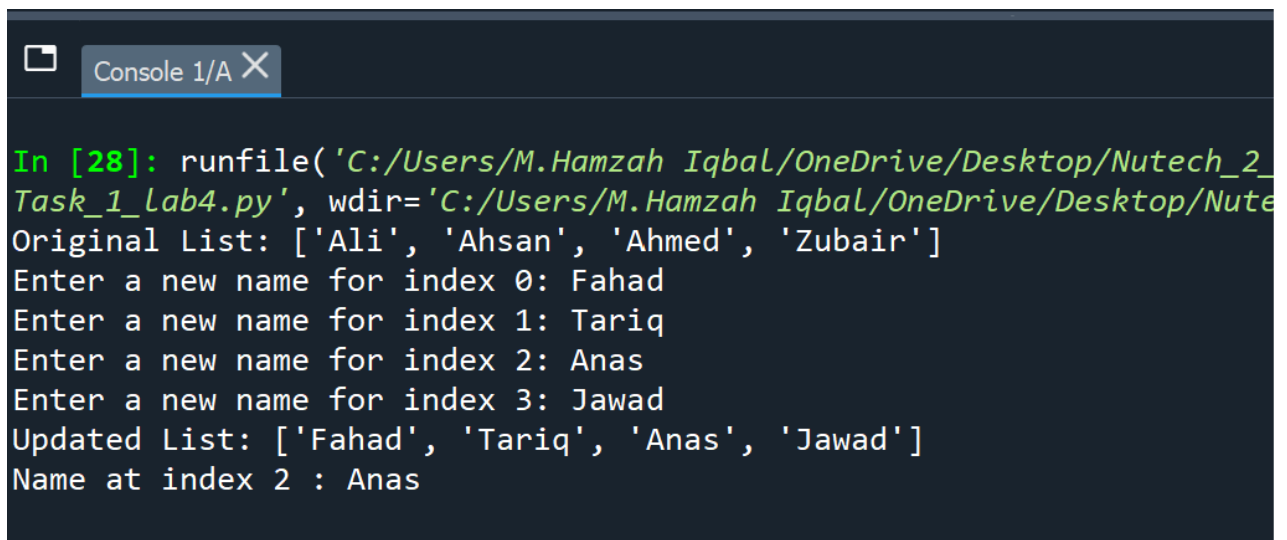


Experimental Result:



```
Task_1_lab4.py X Task_2_lab4.py X Task_3_lab4.py X Task_4_lab4.py X
1 # -*- coding: utf-8 -*-
2 """
3 @author: M.Hamzah Iqbal
4 """
5 Names = ['Ali', 'Ahsan', 'Ahmed', 'Zubair']
6 print("Original List:", Names)
7
8 for i in range(len(Names)):
9     Names[i] = input(f"Enter a new name for index {i}: ")
10
11 print("Updated List:", Names)
12
13 name_at_index_2 = Names[-2]
14 print("Name at index 2 :", name_at_index_2)
15
```

Figure 1 Task 1 Input



```
Console 1/A X
In [28]: runfile('C:/Users/M.Hamzah Iqbal/OneDrive/Desktop/Nutech_2_
Task_1_Lab4.py', wdir='C:/Users/M.Hamzah Iqbal/OneDrive/Desktop/Nute
Original List: ['Ali', 'Ahsan', 'Ahmed', 'Zubair']
Enter a new name for index 0: Fahad
Enter a new name for index 1: Tariq
Enter a new name for index 2: Anas
Enter a new name for index 3: Jawad
Updated List: ['Fahad', 'Tariq', 'Anas', 'Jawad']
Name at index 2 : Anas
```

Figure 2 Output of Task

Task 02

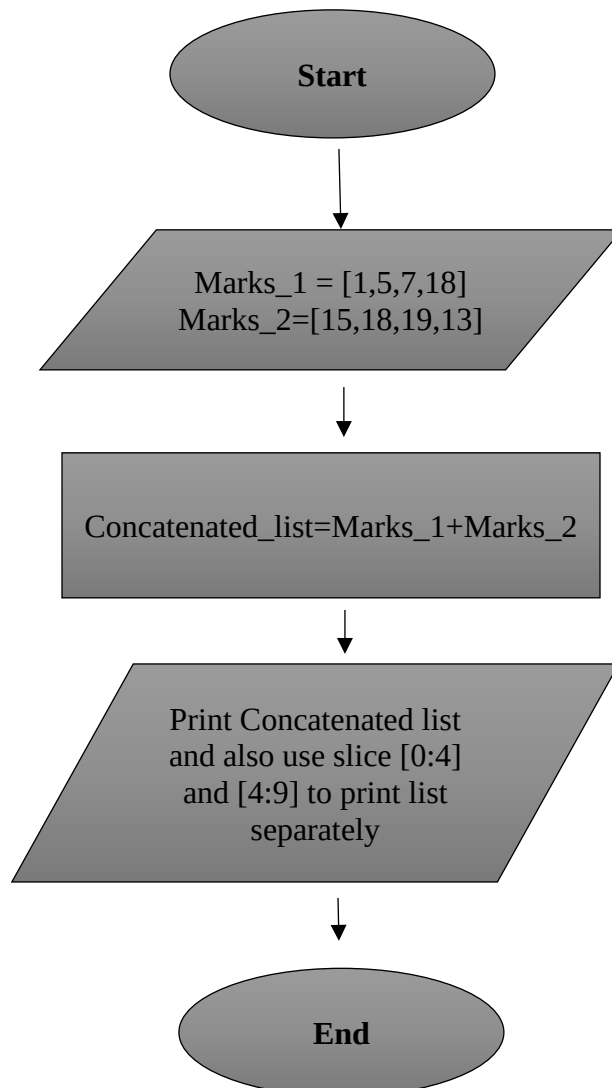
Create two lists; each contains marks of four subjects. Concatenate them and print the result list. Then slice them to recover the original lists and print them too.

Solution:

Algorithm:

1. Create two lists: marks1 and marks2, each containing marks for four subjects.
2. Concatenate the two lists into a new list combined_marks.
3. Print the concatenated list.
4. Slice combined_marks to recover marks1 and marks2.
5. Print the original lists.

Flowchart:



Experimental Results:

```
1 # -*- coding: utf-8 -*-
2 """
3 Created on Wed Mar 19 19:29:01 2025
4
5 @author: M.Hamzah Iqbal
6 """
7 marks_1=[12,14,15,16]
8 marks_2=[18,19,20,21]
9
10 Final_list=marks_1+marks_2
11
12 print("Concatenated List: ",Final_list,"\n")
13 print("Now printing Sliced List : \n")
14 print("    MARKS 1 list : ",Final_list[0:4])
15 print("    MARKS 2 list : ",Final_list[4:9])
16
```

Figure 3 Task 2 Input

```
In [29]: runfile('C:/Users/M.Hamzah Iqbal/OneDrive/Desktop/
Task_2_Lab4.py', wdir='C:/Users/M.Hamzah Iqbal/OneDrive/D
Concatenated list: [12, 14, 15, 16, 18, 19, 20, 21]

Now printing Sliced List :

    MARKS 1 list : [12, 14, 15, 16]
    MARKS 2 list : [18, 19, 20, 21]
```

Figure 4 Task 2 Output

Task 03

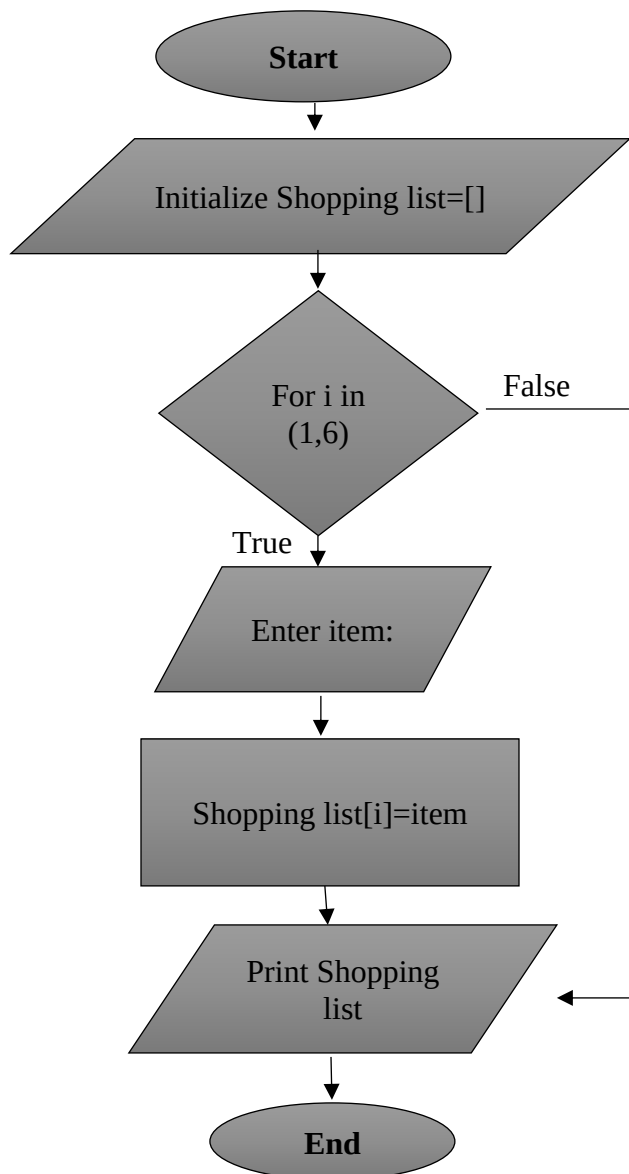
Create a program that will keep track of items for a shopping list. The program should ask 5 times for new items and then display the full shopping list.

Solution:

Algorithm:

1. Initialize an empty list called shopping_list.
2. Loop 5 times:
 - Prompt the user to enter a new item.
 - Add the item to shopping_list.
3. Print the full shopping_list

Flowchart:



Experimental Results:

```
1 # -*- coding: utf-8 -*-
2 """
3 @author: M.Hamzah Iqbal
4 """
5 Shopping_list=[]
6 i=1
7 for i in range(1,6):
8     item=input(f"Enter item {i} in shopping list : ")
9     Shopping_list.append(item)
10
11 print(Shopping_list)
12
13 Shopping_list.clear()
14
15
```

Figure 5 Task 3 Input

```
In [30]: runfile('C:/Users/M.Hamzah Iqbal/OneDrive/Desktop/
Task_3_Lab4.py', wdir='C:/Users/M.Hamzah Iqbal/OneDrive/Des
Enter item 1 in shopping list : Shoes
Enter item 2 in shopping list : Cooking Oil
Enter item 3 in shopping list : Youghurt
Enter item 4 in shopping list : Eggs
Enter item 5 in shopping list : Banana
['Shoes', 'Cooking Oil', 'Youghurt', 'Eggs', 'Banana']
```

Figure 6 Task 3 Output

Task 04

Make a list of ten numbers and find out following and print.

- 1.1. The length of list**
- 1.2. Max number from the list**
- 1.3. Min number from list**
- 1.4. Difference between the max and min**
- 1.5. Sum of list.**
- 1.6. Average of the list.**
- 1.7. Sort the array.**
- 1.8. Find out the index of max number and delete it**

Solution:

Algorithm:

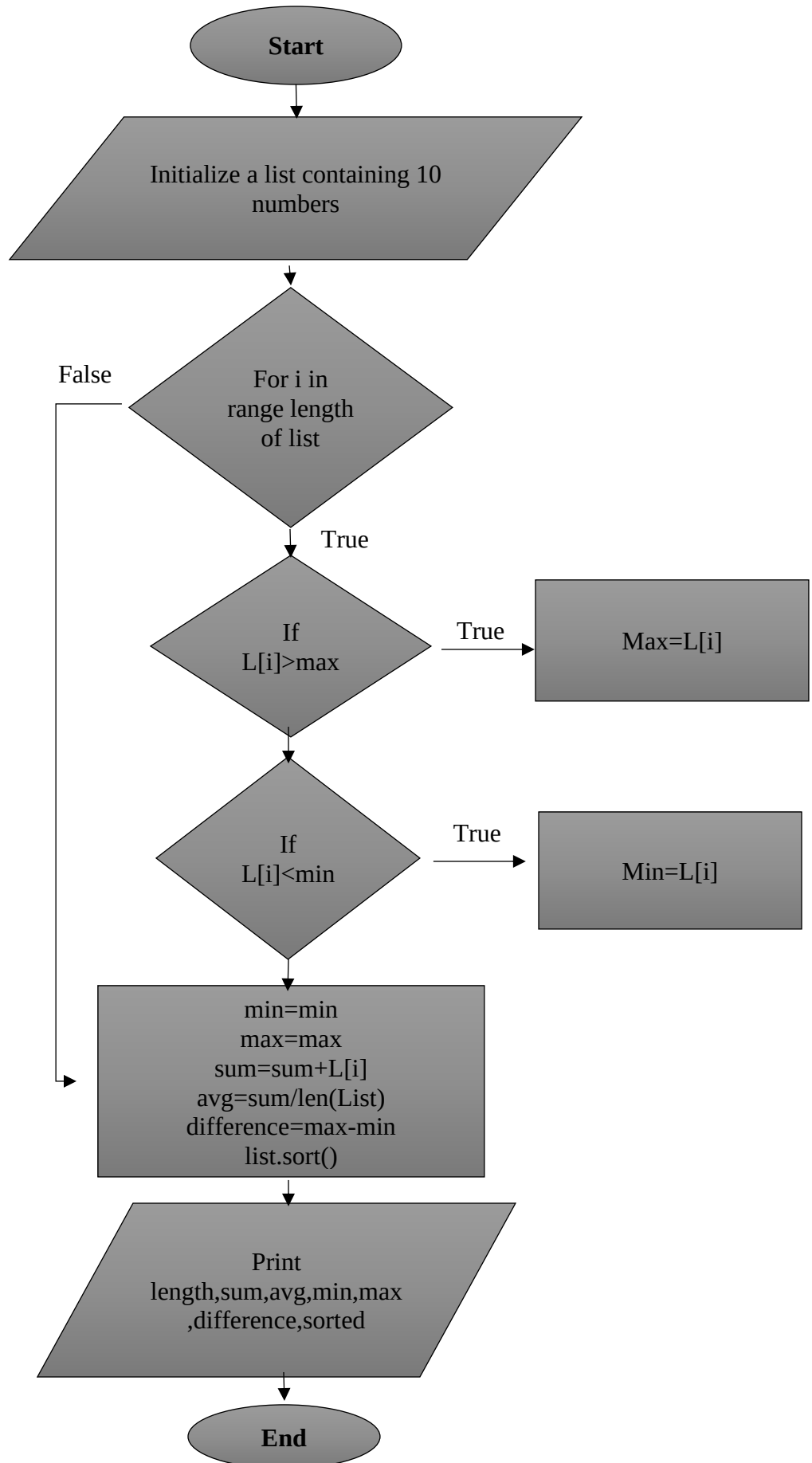
1. Create a list of ten numbers.
2. Calculate and print:
 - The length of the list.
 - The maximum number from the list.
 - The minimum number from the list.
 - The difference between the maximum and minimum numbers.
 - The sum of the list.
 - The average of the list.
3. Sort the list and print the sorted list.
4. Find the index of the maximum number, delete it from the list, and print the updated list.

Pseudocode:

1. Initialize any numbers = [12, 45, 23, 67, 34, 89, 10, 56, 78, 90]
 - length_of_list = LENGTH(numbers)
 - max_number = MAX(numbers)
 - min_number = MIN(numbers)
 - difference = max_number - min_number
 - sum_of_list = SUM(numbers)
 - average = sum_of_list / length_of_list
 - sorted_list = SORT(numbers)
 - index_of_max = INDEX(numbers, max_number)
 - DELETE numbers[index_of_max]

PRINT length_of_list, max_number, min_number, difference, sum_of_list, average,
sorted_list,numbers

Flowchart:



Experimental Results

```
1 # -*- coding: utf-8 -*-
2 """
3 @author: M.Hamzah Iqbal
4 """
5 Numbers_list=[1,4,5,8,19,23,43,22,5,18]
6 length=len(Numbers_list)
7 print("Original List : ",Numbers_list)
8 print("1. Length of List : ",length)
9 max=Numbers_list[1]
10 min=Numbers_list[1]
11 sum=0
12 for i in range(len(Numbers_list)):
13     if Numbers_list[i]>max:
14         max=Numbers_list[i]
15
16     elif Numbers_list[i]<min:
17         min=Numbers_list[i]
18
19     sum=sum+Numbers_list[i]
20     avg=sum/len(Numbers_list)
21
22 print("2.Maximum value in List : ",max)
23 print("3.Minimum value in List : ",min)
24 print("4.Difference between Max and Min : ",max-min)
25 print("5.Sum of List :",sum)
26 print("6.Average : ",avg)
27
28 Numbers_list.sort()
29 print("7.Sorted List : ",Numbers_list)
30
31 max_index = Numbers_list.index(max)
32 del Numbers_list[max_index]
33
```

Figure 7 Task 4 Input

```
Original List : [1, 4, 5, 8, 19, 23, 43, 22, 5, 18]
1. Length of list : 10
2.Maximum value in List : 43
3.Minimum value in List : 1
4.Difference between Max and Min : 42
5.Sum of list : 148
6.Average : 14.8
7.Sorted list : [1, 4, 5, 5, 8, 18, 19, 22, 23, 43]
```

Figure 8 Task 4 Output

2.What will be the output of the following code snippet?

1. `a=[1,2,3,4,5,6,7,8,9]`
`print(a[::-2])`

Output:

[1, 3, 5, 7, 9] ## Selects [n+2] element

2. `a=[1,2,3,4,5,6,7,8,9]`
`a[::-2]=10,20,30,40,50,60`
`print(a)`

Output:

Error, Since there are 6 values and 5 positions

3. `a=[1,2,3,4,5]`
`print(a[3:0:-1])`

Output:

[4, 3, 2] Prints element 1,2,3 using negative indexing

4. `def f(value, values):`
 `v = 1`
 `values[0] = 44`
 `t = 3`
 `v = [1, 2, 3]`
`f(t, v) print(t, v[0])`

Output:

Error, since t and v are not defined

5. `data = [[[1, 2], [3, 4]], [[5, 6], [7, 8]]]`
`def fun(m):`
 `v = m[0][0]`

 `for row in m:`
 `for element in row:`
 `if v < element: v = element`

 `return v`
`print(fun(data[0])`
`)`

Output:

Function finds the maximum value in [[1, 2], [3, 4]] which is : 4

6. `arr = [[1, 2, 3, 4],`

 `[4, 5, 6, 7],`
 `[8, 9, 10, 11],`
 `[12, 13, 14, 15]]`
 `for i in range(0, 4):`
 `print(arr[i].pop())`

Output:

4 # pop() function returns last value
7
11
15

Discussion of Results

In this lab we achieved the following:

- Demonstrated key list operations: indexing, slicing, concatenation, and modification.
- Successfully overwrote list elements and converted the first three names to uppercase.
- Concatenated two lists and recovered original lists using slicing.
- Analyzed a list of ten numbers to find length, max/min values, sum, average, and sorting.
- Executed code snippets to understand list slicing behavior, function parameter passing, and nested lists.
- Encountered and analyzed assignment errors in slicing, reinforcing Python's list constraints.

Conclusions

- Python lists are flexible and versatile, supporting dynamic modifications.
- Lists allow heterogeneous data storage and advanced indexing techniques.
- Challenges like assignment mismatches in slicing highlight the need for careful data handling.
- Understanding list operations is essential for efficient programming and data manipulation.